

Atlas v1.6.4 upgrade

Security Review

Solo review by:

Optimum, Lead Security Researcher

October 14, 2025

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Informational	4
3.1.1	Wrong in-line comment of the ArbSys code	4
3.1.2	Native token refunds to the <code>msg.sender</code> are discouraged	4
3.1.3	Signature verification ignores returned error codes	4

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A security review is a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While the review endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that a security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Atlas is a generalized Execution Abstraction protocol developed by FastLane Labs that aims to reduce the complexity and cost associated with deploying application-specific order flow auctions (OFAs) while optimizing transaction processing and improving value capture within blockchain ecosystems.

From Oct 9th to Oct 10th the security researchers conducted a review of [Fastlane - v1.6.4 upgrade](#) on commit hash [c76196a8](#). A total of **3** issues were identified:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	3	2	1
Total	3	2	1

3 Findings

3.1 Informational

3.1.1 Wrong in-line comment of the `ArbSys` code

Severity: Informational

Context: [SafeBlockNumber.sol#L7](#)

Description: The `ArbSys` contract is a pre-compile contract deployed on the arbitrum chain. It is not implemented in Solidity and therefore the code that resides there is just a placeholder. It is implemented in Go instead as part of the nitro repo.

Recommendation: Consider changing the comment to refer to `$HYPERLINK0` instead.

Fastlane: Fixed in PR 3.

Optimum: Fix verified.

3.1.2 Native token refunds to the `msg.sender` are discouraged

Severity: Informational

Context: [Atlas.sol#L109](#)

Description: The `metacall()` function currently refunds the bundler by sending any leftover native tokens back to `msg.sender`. However, this approach assumes that the sender is capable of receiving native tokens, which might not always be the case - for instance, if the sender is a contract that does not implement a payable fallback function.

Recommendation: Consider allowing the caller to specify a dedicated refund recipient address, similar to the `gasRefundBeneficiary()` pattern. This would make refunds more flexible and prevent potential failures when sending native tokens. However, note that changing the refund mechanism could impact backward compatibility with existing integrations that rely on the current behavior. Carefully assess this trade-off before implementation.

Fastlane: Acknowledging the issue, prefers to improve documentation for bundlers instead of implementing the recommended fix.

Optimum: Acknowledged.

3.1.3 Signature verification ignores returned error codes

Severity: Informational

Context: [AtlasVerification.sol#L378](#)

Description: Both `_verifySolverSignature()` and `_verifyDAppSignature()` utilize `ECDSA.tryRecover()`, which returns an error code along with `address(0)` when the signature is invalid. However, both functions currently ignore the returned error code. While the impact is limited - since `_verifyDAppSignature()` will eventually revert due to the `address(0)` result - the handling in `_verifySolverSignature()` is unclear, as it was outside the scope of this engagement.

Recommendation: Explicitly handle the error code returned by `tryRecover()` and revert in case of any error. This would improve code clarity and ensure consistent behavior across all signature verification paths.

Fastlane: Fixed in PR 1.

Optimum: Fix verified.